

AI로 코드 고치기

Copilot · ChatGPT · Claude



개발을 몰라도 됩니다. AI에게 원하는 것을 말로 설명하면 코드를 고칠 수 있습니다. 흔히 "바이브 코딩"이라 부르는 방식입니다.

GitHub 안에서 수정부터 빌드까지

파일을 복사해 옮길 필요 없이, GitHub 안에서 Copilot이 코드를 고치고, 저장하면 앱(APK)까지 자동으로 만들어집니다. 개발을 몰라도 **말로 설명**하면 됩니다.

01 GitHub에서 Copilot 열기

저장소 화면에서 **Copilot**(github.com/copilot)을 엽니다. 편집기를 따로 설치하지 않아도 GitHub 안에서 바로 쓸 수 있습니다. (Copilot 요금제 필요 — 무료 플랜 있음)

02 고칠 저장소·파일 지정

대화창에 저장소(또는 파일)를 **첨부**하고, 원하는 것을 말로 적습니다. 예: "index.html에서 로그인 버튼 색을 초록색으로 바꿔줘."

03 Copilot이 코드 수정

Copilot이 고친 코드(또는 파일)를 만들어 줍니다. 옆 패널에서 **보기·수정·내려받기**할 수 있습니다. 더 나아가 Copilot이 변경 사항을 **브랜치에 반영**하고 **Pull Request**까지 열어 줄 수도 있습니다.

04 변경 확인하고 저장(병합)

바뀐 내용(diff)을 확인하고 **Commit** 또는 **Merge**(병합)합니다. Copilot이 커밋 메시지도 제안해 줍니다.

05 빌드 결과 확인 (APK)

코드가 저장되면 GitHub가 자동으로 앱을 만듭니다. 저장소 상단 **Actions** 탭에서 진행 상황을 보고, 초록색 체크가 뜨면 **Artifacts**에서 완성된 APK를 내려받습니다.

Claude · ChatGPT 커넥터로 연결하기

Claude와 ChatGPT도 GitHub 저장소를 **연결**해, 코드를 읽고 수정안을 만들어 줍니다. 다만 대부분의 커넥터는 **읽기 전용**이라, 완성된 수정은 사용자가 GitHub에 반영(붙여넣기·커밋)합니다.

C Claude — 커넥터에 GitHub 추가

Claude의 **+ 버튼 → GitHub에서 추가** 또는 **설정 → 커넥터**에서 저장소를 연결합니다. Claude가 저장소를 읽고, "이 부분을 이렇게 바꿔줘"라고 하면 고친 코드를 만들어 줍니다. 그 코드를 GitHub 파일의 연필(Edit)에 붙여넣고 **Commit**합니다.

G ChatGPT — 커넥터 / 에이전트

ChatGPT의 **설정 → 앱(커넥터)**에서 GitHub를 연결하면 저장소를 읽고 분석해 줍니다. 코드를 **직접 생성·수정·반영**하려면 OpenAI의 **Codex**(소프트웨어 엔지니어링 에이전트)를 사용합니다. Codex는 코드·PR·워크플로에 반영까지 할 수 있습니다.

• 복사·붙여넣기 방법(가장 간단)

연결이 어려우면, GitHub에서 파일을 복사해 Claude/ChatGPT에 붙여넣고 수정을 요청한 뒤, 돌려받은 코드를 다시 붙여넣어 **Commit**해도 됩니다.

읽기와 반영은 다릅니다

대부분의 커넥터(Claude·ChatGPT)는 저장소를 **읽어서** 수정안을 만들어 줍니다. GitHub에 **직접 반영(push)**하는 것은 별도 기능입니다 — GitHub Copilot, 또는 ChatGPT의 Codex 같은 에이전트가 그 역할을 합니다.

저작권 · 라이선스 주의

AI로 만든·고친 코드의 저작권은 아직 정리되어 가는 영역입니다. 외부 라이브러리 라이선스 조건을 지켜야 하며, 공개·배포 전에는 저작권·라이선스 사항을 확인해 두는 것이 좋습니다.

입력 문구와 화면 안내

입력하는 문구와, 이후 나타나는 버튼·메시지를 예시로 정리했습니다. (도구·버전에 따라 문구는 조금씩 다를 수 있습니다.)

1 입력 문구

AI 대화창에 다음과 같이 입력합니다:

- "index.html에서 로그인 버튼 색을 초록색으로 **바꿔줘**."
- "이 변경을 저장소에 **반영해줘**." (Copilot·Codex 등 반영 가능한 도구일 때)

2 표시되는 메시지

- 고친 코드가 표시되고, 아래에 **"Apply"(적용)** / **"Commit"(저장)** 버튼이 나타납니다.
- 반영형 도구는 **"Create pull request"(PR 만들기)** 또는 **"Changes committed"(변경 저장됨)** 메시지를 표시합니다.

3 확인 요청

- **"이 변경을 적용할까요?"** 또는 **"Approve / Confirm"** 같은 확인 버튼이 나오면 눌러 승인합니다.
- 읽기 전용 도구(대부분의 커넥터)는 "직접 반영은 지원하지 않는다"는 안내가 나올 수 있고, 이때는 코드를 복사해 GitHub에 붙여넣습니다.

4 빌드 시작

저장(Commit/Merge)되면 GitHub **Actions** 탭에 **"Build Android APK"** 작업이 나타나고, 노란 점(진행) → 초록 체크(완료)로 바뀝니다.

5 결과 내려받기

완료되면 그 작업 화면 아래 **"Artifacts"**에 **"MediNote-debug-apk"**가 나옵니다. 눌러서 내려받으면 APK 파일을 얻습니다.

자주 쓰는 입력 문구

· "글자를 더 크게 **바꿔줘**" · "네이버 로그인 버튼을 **추가해줘**" · "이 오류를 **고쳐줘**" · "방금 변경을 저장소에 **반영해줘**" · "무엇이 바뀌었는지 **설명해줘**"